

Examen Práctico: API REST de Gestión de Pedidos

1. Contexto del caso

Una pequeña tienda necesita una API REST que permita registrar clientes, productos y pedidos. Cada pedido puede tener varios productos asociados, por lo tanto, el sistema debe manejar una relación de **uno a muchos**.

deberá construir una API usando:

- Java 21
 - Maven
 - Spring Boot
 - PostgreSQL
 - Spring Data JPA
 - Validaciones
 - Pruebas unitarias
 - Buenas prácticas de arquitectura
 - Principios SOLID
 - Patrón Builder
 - Manejo de excepciones personalizadas
-

2. Caso a desarrollar

Se debe construir una API llamada:

API de Gestión de Pedidos para una Tienda

La API permitirá:

1. Registrar productos.
 2. Registrar clientes.
 3. Crear pedidos para un cliente.
 4. Agregar varios productos dentro de un pedido.
 5. Consultar un pedido por ID.
 6. Listar los pedidos de un cliente.
 7. Manejar errores mediante excepciones personalizadas.
 8. Retornar todas las respuestas usando una clase BaseResponse.
-

3. Modelo de datos obligatorio

debe implementar al menos las siguientes entidades:

Cliente

Campos sugeridos:

- id
- nombre
- apellido
- dni
- correo
- fechaRegistro

Relación:

Un cliente puede tener muchos pedidos.

```
@OneToMany(mappedBy = "cliente")  
private List<Pedido> pedidos;
```

Pedido

Campos sugeridos:

- id
- fechaPedido
- estado
- total
- cliente

Relación:

Muchos pedidos pertenecen a un cliente.

```
@ManyToOne  
@JoinColumn(name = "cliente_id")  
private Cliente cliente;
```

Además, un pedido puede tener muchos detalles.

```
@OneToMany(mappedBy = "pedido", cascade = CascadeType.ALL)  
private List<DetallePedido> detalles;
```

DetallePedido

Campos sugeridos:

- id
- productoId
- nombreProducto
- cantidad
- precioUnitario
- subtotal
- pedido

Relación:

Muchos detalles pertenecen a un pedido.

```
@ManyToOne
@JoinColumn(name = "pedido_id")
private Pedido pedido;
```

Producto

Campos sugeridos:

- id
 - nombre
 - descripcion
 - precio
 - stock
 - estado
-

4. BaseResponse obligatorio

Todas las respuestas de la API deben devolver la siguiente estructura:

```
public class BaseResponse<T> {

    private Integer codigo;
    private String mensaje;
    private T objeto;

}
```

Ejemplo de respuesta correcta:

```
{
  "codigo": 200,
  "mensaje": "Pedido creado correctamente",
  "objeto": {
    "id": 1,
    "cliente": "Juan Pérez",
    "total": 150.50,
    "estado": "CREADO"
  }
}
```

Ejemplo de respuesta con error:

```
{
  "codigo": 404,
  "mensaje": "No se encontró el pedido solicitado",
  "objeto": null
}
```

5. Excepciones personalizadas obligatorias

El proyecto debe tener como mínimo dos excepciones personalizadas.

Excepción 1: PedidoNotFoundException

Se debe lanzar cuando se intente buscar un pedido que no existe.

Ejemplo:

```
public class PedidoNotFoundException extends RuntimeException {

    public PedidoNotFoundException(String mensaje) {
        super(mensaje);
    }
}
```

Excepción 2: StockInsuficienteException

Se debe lanzar cuando el cliente intente comprar una cantidad mayor al stock disponible.

Ejemplo:

```
public class StockInsuficienteException extends RuntimeException {  
  
    public StockInsuficienteException(String mensaje) {  
        super(mensaje);  
    }  
}
```

6. Reglas de negocio obligatorias

El servicio debe cumplir las siguientes reglas:

Crear pedido

Para crear un pedido:

1. El cliente debe existir.
 2. Cada producto enviado en el pedido debe existir.
 3. La cantidad solicitada debe ser mayor a cero.
 4. El producto debe tener stock suficiente.
 5. El sistema debe calcular el subtotal de cada detalle.
 6. El sistema debe calcular el total del pedido.
 7. Al crear el pedido, se debe descontar el stock de cada producto.
 8. El pedido debe quedar con estado CREADO.
-

7. Endpoints mínimos esperados

ProductoController

Crear producto

POST /api/productos

Body ejemplo:

```
{  
  "nombre": "Teclado mecánico",  
  "descripcion": "Teclado RGB",  
  "precio": 150.00,  
  "stock": 20  
}
```

Listar productos

GET /api/productos

ClienteController

Crear cliente

POST /api/clientes

Body ejemplo:

```
{
  "nombre": "Juan",
  "apellido": "Pérez",
  "dni": "12345678",
  "correo": "juan.perez@gmail.com"
}
```

Buscar cliente por ID

GET /api/clientes/{id}

PedidoController

Crear pedido

POST /api/pedidos

Body ejemplo:

```
{
  "clienteId": 1,
  "items": [
    {
      "productoId": 1,
      "cantidad": 2
    },
    {
      "productoId": 2,
      "cantidad": 1
    }
  ]
}
```

Buscar pedido por ID

GET /api/pedidos/{id}

Listar pedidos por cliente

GET /api/pedidos/cliente/{clienteId}

8. Uso obligatorio del patrón Builder

debe aplicar el patrón Builder como mínimo en una de estas clases:

- Pedido
- DetallePedido
- Producto
- BaseResponse

Puede usar Builder manual o Lombok.

Ejemplo usando Lombok:

```
@Builder
@Getter
@Setter
@AllArgsConstructor
@NoArgsConstructor
public class BaseResponse<T> {

    private Integer codigo;
    private String mensaje;
    private T objeto;

}
```

Ejemplo de uso:

```
return BaseResponse.<PedidoResponse>builder()
    .codigo(200)
    .mensaje("Pedido creado correctamente")
    .objeto(response)
    .build();
```

9. Aplicación de principios SOLID

El proyecto debe evidenciar buenas prácticas de diseño.

S - Single Responsibility Principle

Cada clase debe tener una sola responsabilidad.

Ejemplo:

- PedidoController: recibe peticiones HTTP.
 - PedidoService: contiene la lógica de negocio.
 - PedidoRepository: accede a base de datos.
 - PedidoMapper: transforma entidades a DTOs.
-

O - Open/Closed Principle

La lógica debe permitir extender el comportamiento sin modificar demasiado el código existente.

Ejemplo:

- Usar interfaces como PedidoService.
 - Implementar la lógica en PedidoServiceImpl.
-

D - Dependency Inversion Principle

El servicio no debe depender directamente de implementaciones concretas.

Ejemplo:

```
private final PedidoRepository pedidoRepository;  
private final ProductoRepository productoRepository;  
private final ClienteRepository clienteRepository;
```

Estas dependencias deben inyectarse por constructor.

10. Pruebas unitarias obligatorias

debe crear pruebas unitarias con JUnit 5 y Mockito.

Como mínimo debe tener:

Happy path

Prueba exitosa para crear un pedido.

Nombre sugerido:

`crearPedido_cuandoDatosSonValidos_retornaPedidoCreado()`

Debe validar:

- Que el cliente existe.
 - Que los productos existen.
 - Que el stock es suficiente.
 - Que el total se calcula correctamente.
 - Que se guarda el pedido.
 - Que retorna una respuesta correcta.
-

Error path

Prueba de error cuando no hay stock suficiente.

Nombre sugerido:

`crearPedido_cuandoStockEsInsuficiente_lanzaStockInsuficienteException()`

Debe validar:

- Que se lanza `StockInsuficienteException`.
 - Que el pedido no se guarda.
 - Que el stock no se descuenta.
-

Error path adicional sugerido

Prueba cuando no existe el pedido.

Nombre sugerido:

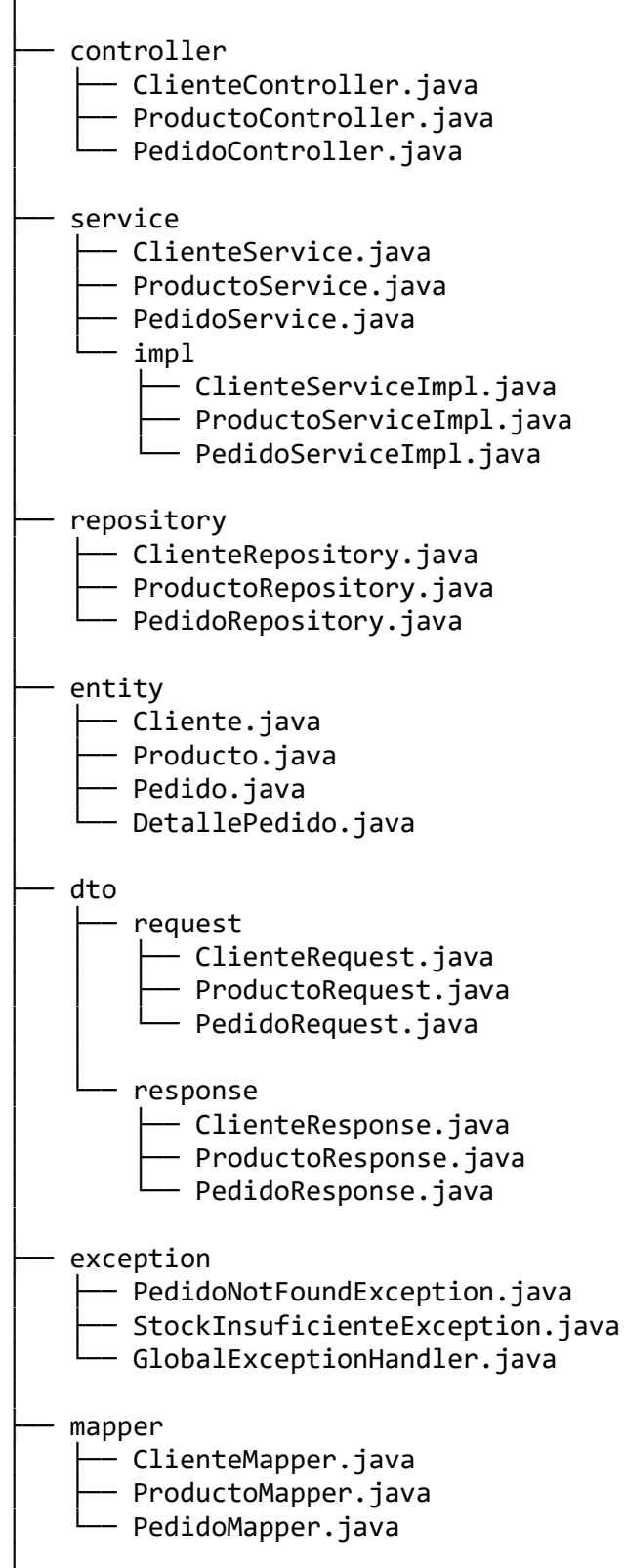
`buscarPedido_cuandoNoExiste_lanzaPedidoNotFoundException()`

Debe validar:

- Que se lanza `PedidoNotFoundException`.
-

11. Estructura sugerida del proyecto

src/main/java/com/examen/pedidos



└─ response
└─ BaseResponse.java

12. Configuración obligatoria de PostgreSQL

El proyecto debe usar PostgreSQL.

Ejemplo de configuración en `application.properties`:

```
spring.datasource.url=jdbc:postgresql://localhost:5432/db_pedidos
spring.datasource.username=postgres
spring.datasource.password=admin
```

```
spring.jpa.hibernate.ddl-auto=update
spring.jpa.show-sql=true
spring.jpa.properties.hibernate.format_sql=true
```

13. Entregables del examen

debe entregar:

1. Link del repositorio GitHub con el código fuente.
 2. Proyecto Maven funcional.
 3. Script SQL o instrucciones para crear la base de datos.
 4. Archivo `README.md`.
 5. Evidencia de pruebas unitarias ejecutadas.
 6. Capturas opcionales de pruebas en Postman o Swagger.
-

14. Requisitos mínimos del README.md

El archivo `README.md` debe incluir:

Nombre del proyecto

Ejemplo:

API REST de Gestión de Pedidos

Tecnologías usadas

Ejemplo:

Java 21
Spring Boot
Maven
PostgreSQL
JUnit 5
Mockito

Instrucciones para ejecutar

Ejemplo:

```
mvn clean install  
mvn spring-boot:run
```

Configuración de base de datos

Ejemplo:

```
CREATE DATABASE db_pedidos;
```

Endpoints disponibles

Debe indicar los endpoints para que el docente pueda probar la API.

Ejemplo:

```
POST /api/clientes  
POST /api/productos  
POST /api/pedidos  
GET /api/pedidos/{id}  
GET /api/pedidos/cliente/{clienteId}
```

Ejemplos de request JSON

Debe incluir ejemplos de body para crear cliente, producto y pedido.